

Aula 04/15

Escola Tecnológica FPFtech

Curso Técnico: Informática para Internet

Módulo: Testes de Software

Prof. Denis Moraes Guimarães

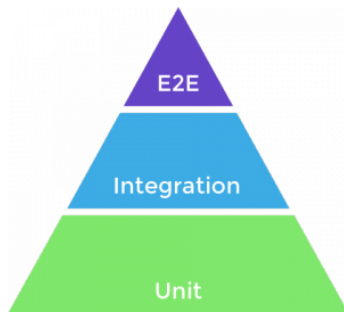
26 de maio de 2026

Sumário

- ▶ Pirâmide de testes e níveis
- ▶ Testes funcionais
- ▶ Testes não funcionais

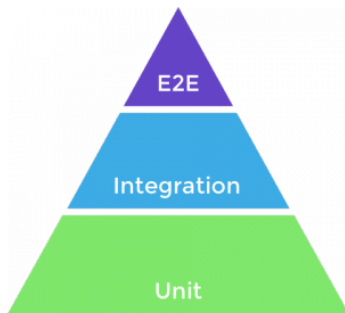
Pirâmide de testes e níveis

- Modelo de estratégia de automação de testes que ajuda a organizar quantos testes fazer em cada nível e onde investir mais esforço.



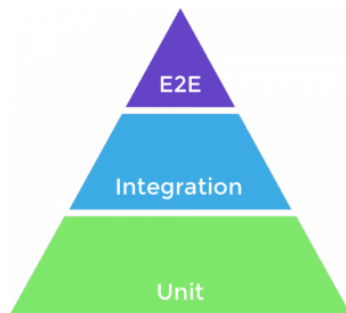
Pirâmide de testes e níveis - Base

- ▶ Base da pirâmide (muitos testes).
- ▶ Testes de unidade.
- ▶ São os mais rápidos, simples e baratos.
- ▶ Testam pequenas partes do código isoladamente (funções, métodos).
- ▶ Devem ser a maioria dos testes.



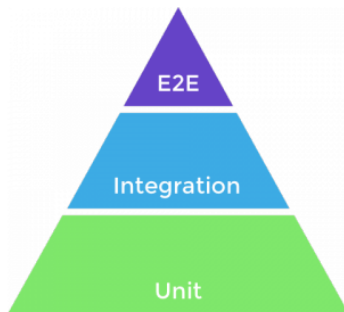
Pirâmide de testes e níveis - Meio

- ▶ Meio da pirâmide (quantidade moderada)
- ▶ Testes de integração.
- ▶ Verificam a comunicação entre módulos (ex: banco de dados + API).
- ▶ São mais lentos e complexos que os de unidade.
- ▶ Existem em menor quantidade.



Pirâmide de testes e níveis - Topo

- ▶ Testes de sistema / ponta a ponta (E2E)
- ▶ Simulam o uso real do sistema do início ao fim.
- ▶ São os mais lentos, frágeis e caros de manter.
- ▶ Devem ser poucos.



Testes funcionais

- ▶ Teste de unidade
- ▶ Teste de integração
- ▶ Teste de sistema
- ▶ Teste de ponta a ponta (E2E)
- ▶ Teste de aceitação (UAT)
- ▶ Teste de regressão
- ▶ Teste de sanidade
- ▶ Teste de fumaça
- ▶ Teste de repetibilidade

Testes funcionais - Unidade

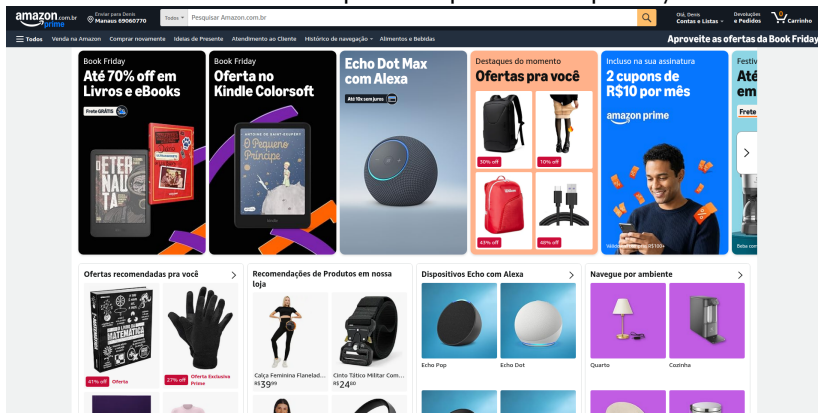
```
1 import unittest
2
3 class TestStringMethods(unittest.TestCase):
4
5     def test_upper(self):
6         self.assertEqual('foo'.upper(), 'FOO')
7
8     def test_isupper(self):
9         self.assertTrue('FOO'.isupper())
10        self.assertFalse('Foo'.isupper())
11
12    def test_split(self):
13        s = 'hello world'
14        self.assertEqual(s.split(), ['hello', 'world'])
15        with self.assertRaises(TypeError):
16            s.split(2)
17
18 if __name__ == '__main__':
19     unittest.main()
```

Testes funcionais - Integração

```
1 import unittest
2 import requests
3
4 class TestHTTTPublishAndResponse(unittest.TestCase):
5     BASE_URL = "http://localhost:5000"
6     def test_publish_and_get_response(self):
7         resp = requests.post(
8             f"{self.BASE_URL}/publish",
9             json={"message": "ping"}
10        )
11
12        self.assertEqual(resp.status_code, 200)
13        resp = requests.get(f"{self.BASE_URL}/response")
14        self.assertEqual(resp.status_code, 200)
15        data = resp.json()
16        self.assertIn("message", data)
17        self.assertEqual(data["message"], "pong")
18
19 if __name__ == "__main__":
20     unittest.main()
```

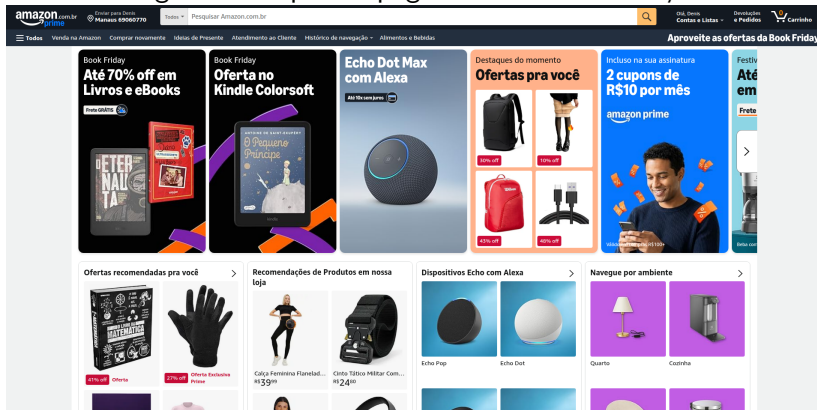
Testes funcionais - Sistema

Ambiente o mais próximo possível da produção



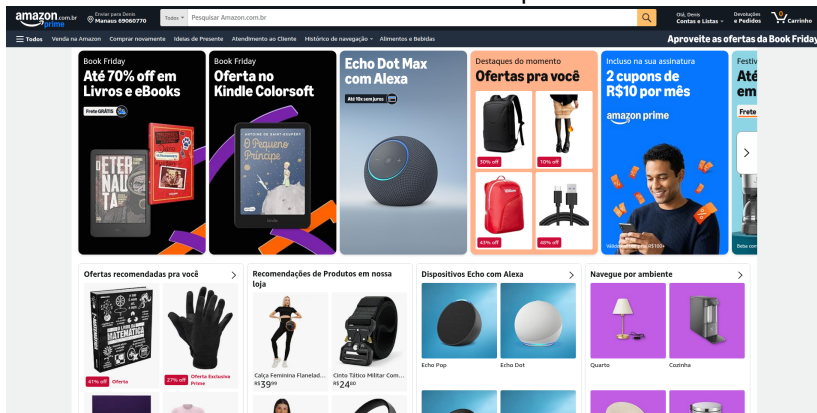
Testes funcionais - Ponta a ponta (E2E)

Login → compra → pagamento → confirmação



Testes funcionais - Aceitação

Validar se o sistema atende aos requisitos do cliente



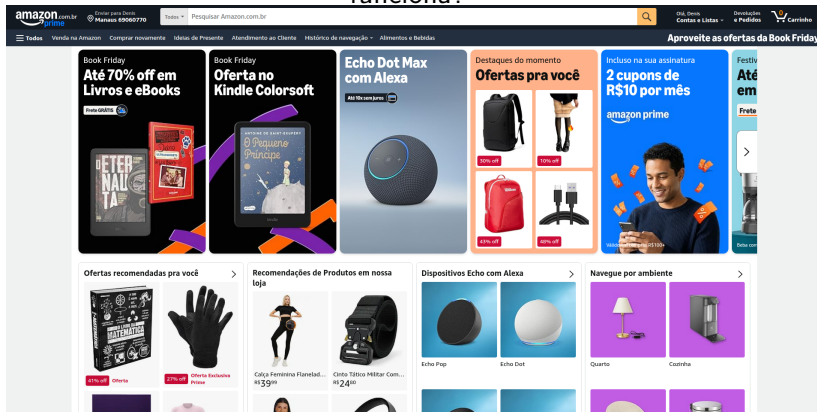
Testes funcionais - Regressão

"Se nova feature for adicionada, o que já existe continua funcionando?"

Status	Pipeline	Created by	Stages	Actions
 Passed 00:43:01 4 days ago	weekly_build_master #397958  master  scheduled latest		    	
 Passed 00:23:27 6 days ago	remove index_matrices #397724  fix_log_features_callback fef22999  latest		 	
 Failed 00:15:42 6 days ago	remove index_matrices #397724  fix_log_features_callback 893035a3 		 	 
 Passed 00:26:02 6 days ago	Merge branch 'fix_logfile_argument' into 'mast... #397563  master  latest		  	
 Passed 00:46:05 6 days ago	replace settings by experiment in log call #397482  fix_logfile_argument 		   	

Testes funcionais - Sanidade

Menos profundo que o regressão, "a parte principal ainda funciona?"



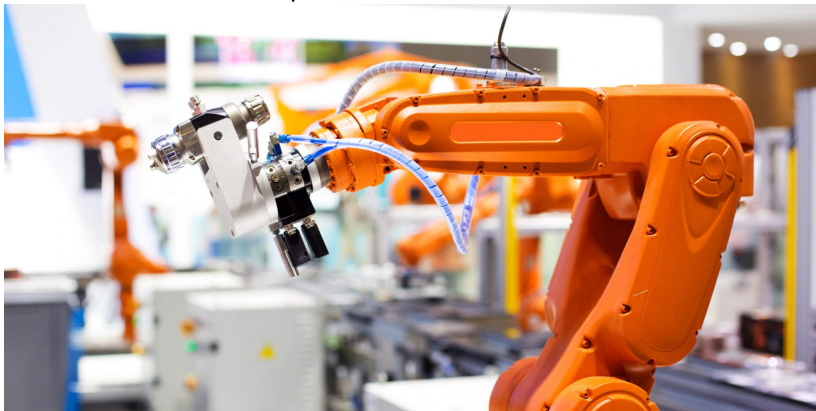
Testes funcionais - Fumaça

Checa se o sistema “sobe” e funciona minimamente.



Testes funcionais - Repetibilidade

"A cada X vezes, o comportamento se mantém Y vezes",
comportamento inconstante.



Testes não funcionais

- ▶ Teste de performance
- ▶ Teste de carga
- ▶ Teste de estresse
- ▶ Teste de escalabilidade
- ▶ Teste de segurança
- ▶ Teste de usabilidade
- ▶ Teste de confiabilidade
- ▶ Teste de disponibilidade
- ▶ Teste de compatibilidade
- ▶ Teste de recuperação
- ▶ Teste de manutenção

Conceitos fundamentais sobre Testes de Software

Teste de Software:

- ▶ **Pergunta-chave:** O sistema funciona corretamente?"
- ▶ **Foco:** detectar defeitos
- ▶ Consiste em executar o software para encontrar erros.
- ▶ **Atividades típicas:** teste unitário, de integração, de sistema, de aceitação;
- ▶ **Exemplo:** executar casos de teste e verificar se a saída está correta.

Conceitos fundamentais sobre Testes de Software

Erro:

- ▶ É a ação humana incorreta.
- ▶ Ocorre quando alguém comete um engano.
- ▶ Está na origem do problema.
- ▶ **Exemplo:** Um dev entende errado um requisito e implementa lógica incorreta.

Conceitos fundamentais sobre Testes de Software

Defeito:

- ▶ É o problema no código ou artefato causado por um erro.
- ▶ Está no software (código, design, documentação)
- ▶ Pode ou não se manifestar em execução.
- ▶ **Exemplo:** Uma condição if escrita errado no código.

Conceitos fundamentais sobre Testes de Software

Falha:

- ▶ É quando o sistema se comporta de forma incorreta em execução.
- ▶ É o efeito visível para o usuário (VOC).
- ▶ Só acontece quando o defeito é ativado.
- ▶ **Exemplo:** O sistema retorna valor errado ou trava ao executar (desastre).

Conceitos fundamentais sobre Testes de Software

Problema da Parada (Halting Problem):

- ▶ **Pergunta:** "Dado um programa qualquer e uma entrada, é possível saber sempre se esse programa vai parar ou rodar para sempre?"
- ▶ **Resposta:** "Não existe um algoritmo geral que resolva isso para todos os casos."

